



University
of Exeter

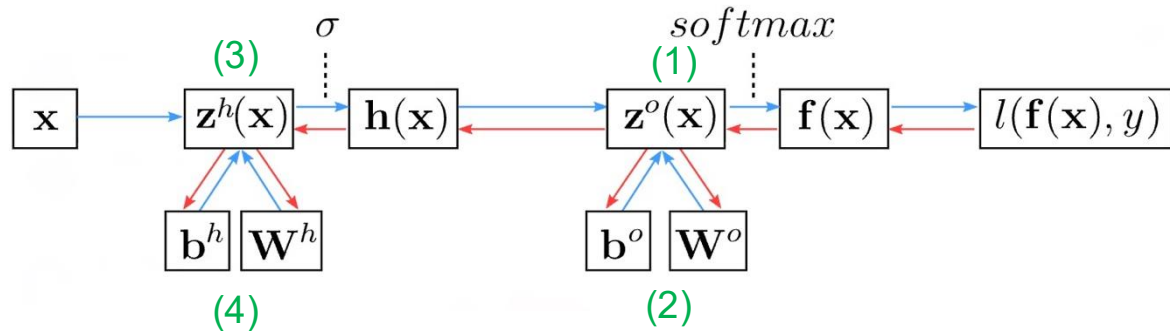
COMM113

Deep Learning – Week 3

Regularisations I

Gaojie (Jay) Jin
g.jin@exeter.ac.uk

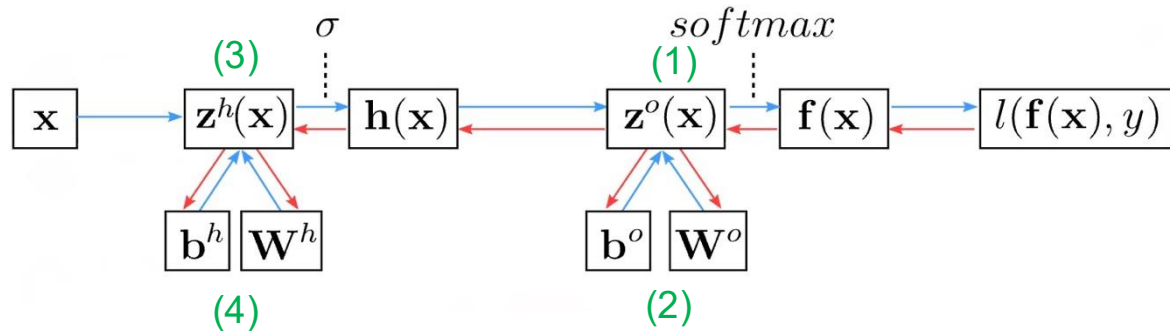
Review - Backpropagation



- $\delta^o = f - y$ (1)
- $\nabla_{W^o} = \delta^o h^\top$, $\nabla_{b^o} = \delta^o$ (2)
- $\delta^h = (W^o)^\top \delta^o \odot \sigma'(z^h)$ (3)
- $\nabla_{W^h} = \delta^h x^\top$, $\nabla_{b^h} = \delta^h$ (4)

⊙ Hadamard product (also known as the element-wise product): multiplying corresponding elements in two matrices

Review - Backpropagation



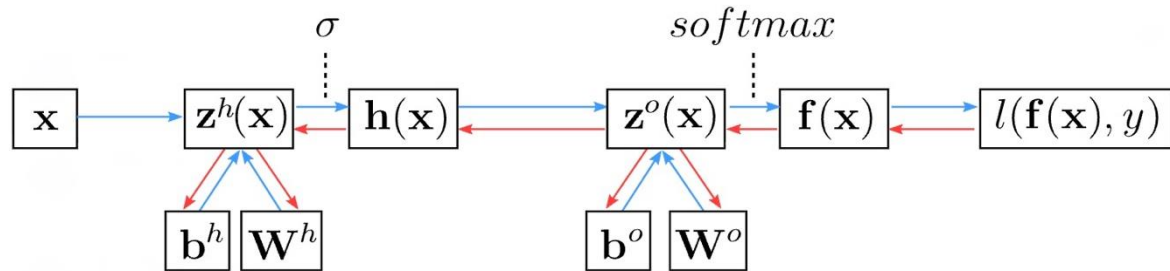
Input and parameters:

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{W}^h = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{b}^h = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\mathbf{W}^o = \begin{bmatrix} 1 & 1 \\ -1 & 2 \end{bmatrix}, \quad \mathbf{b}^o = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Target label $\mathbf{y} = [1, 0]^T$.

Review - Backpropagation



(1) Hidden layer pre-activation:

$$z^h = W^h x + b^h = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 * 1 + (-1) * 2 \\ 0 * 1 + 1 * 2 \end{bmatrix} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$$

(2) Apply ReLU:

$$h = \sigma(z^h) = \max(0, z^h) = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$$

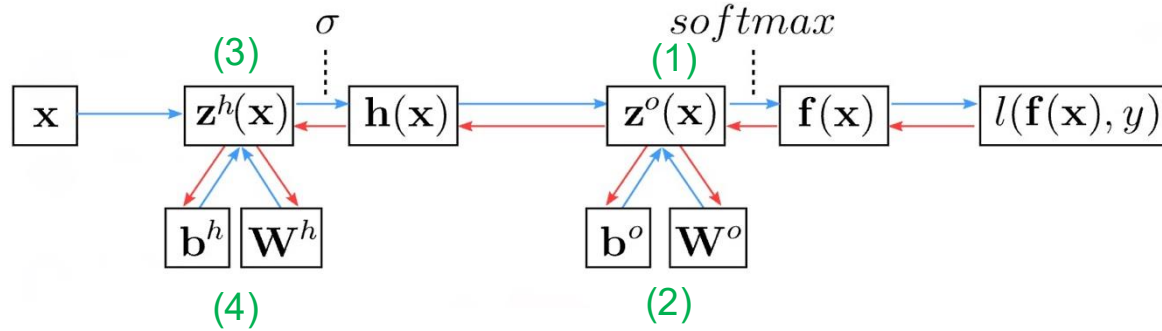
(3) Output layer pre-activation:

$$z^o = W^o h + b^o = \begin{bmatrix} 1 & 1 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 * 0 + 1 * 2 \\ (-1) * 0 + 2 * 2 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

(4) Softmax output:

$$f = \text{softmax}(z^o) = \frac{e^{z^o}}{\sum e^{z^o}} = \frac{1}{e^2 + e^4} \begin{bmatrix} e^2 \\ e^4 \end{bmatrix} = \begin{bmatrix} 0.1192 \\ 0.8808 \end{bmatrix}$$

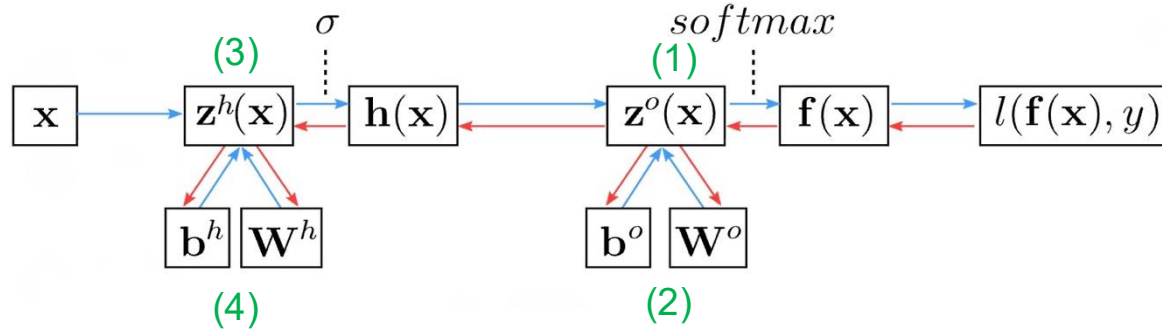
Review - Backpropagation



$\delta^o =$

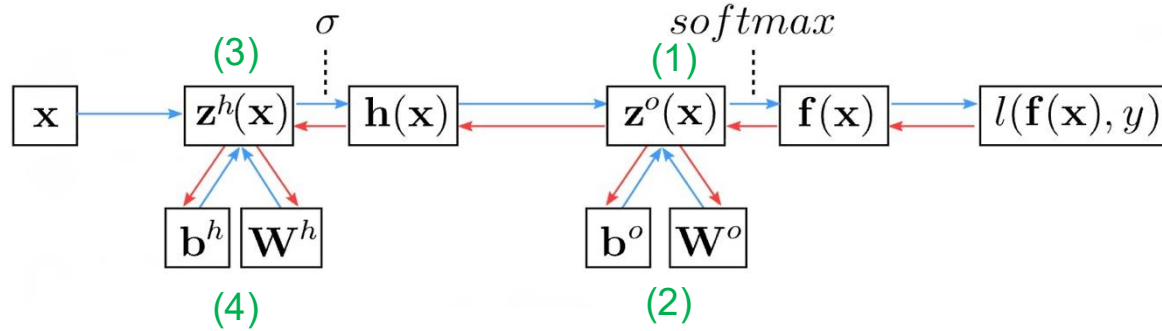
(1) What is the gradient of $\mathbf{z}^o(\mathbf{x})$?

Review - Backpropagation



$$\delta^o = \mathbf{f} - \mathbf{y} = \begin{bmatrix} 0.1192 \\ 0.8808 \end{bmatrix} - \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -0.8808 \\ 0.8808 \end{bmatrix} \quad (1)$$

Review - Backpropagation



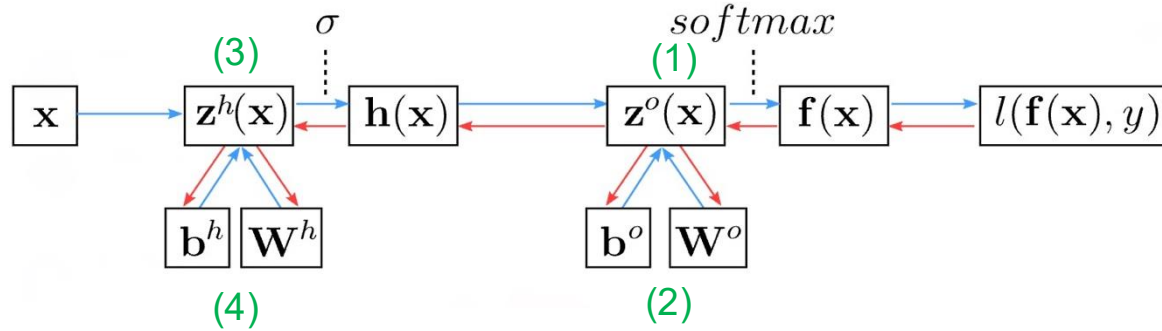
$$\nabla_{W^o} =$$

$$\nabla_{b^o} =$$

What is the gradient of out put weights and bias?

(2)

Review - Backpropagation



$$\nabla_{W^o} = \delta^o h^T, \quad \nabla_{b^o} = \delta^o$$

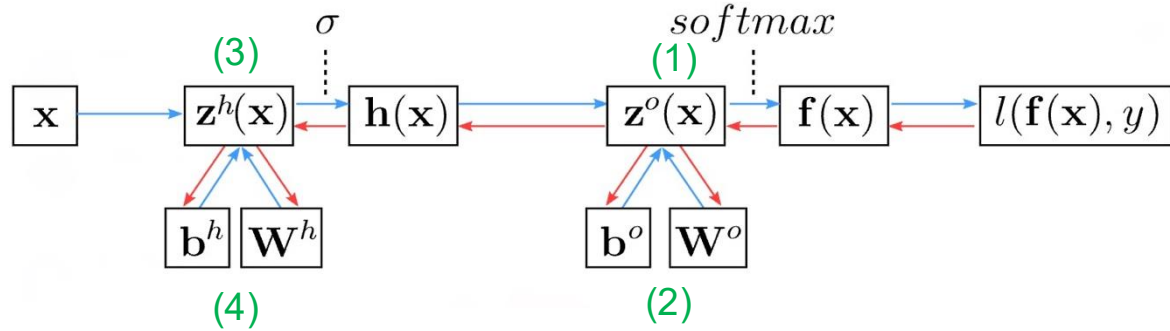
$$h^T = [0, 2]$$

(2)

$$\nabla_{W^o} = \begin{bmatrix} -0.8808 \\ 0.8808 \end{bmatrix} [0, 2] = \begin{bmatrix} -0.8808 * 0 & -0.8808 * 2 \\ 0.8808 * 0 & 0.8808 * 2 \end{bmatrix} = \begin{bmatrix} 0 & -1.7616 \\ 0 & 1.7616 \end{bmatrix}$$

$$\nabla_{b^o} = \begin{bmatrix} -0.8808 \\ 0.8808 \end{bmatrix}$$

Review - Backpropagation

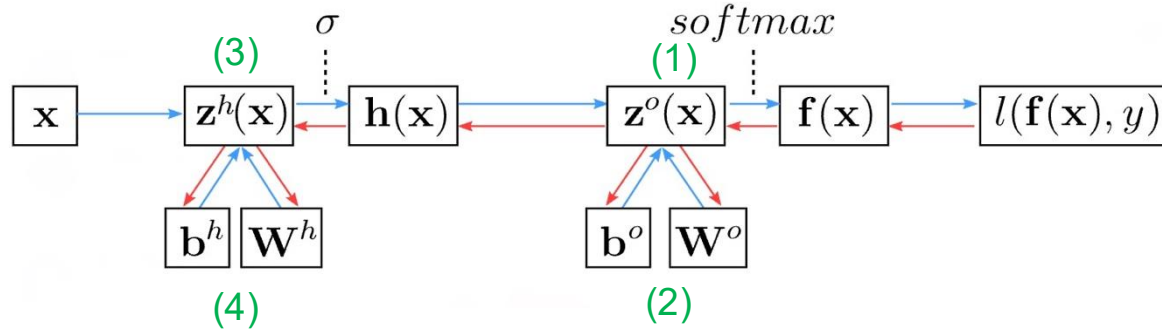


$\delta^h =$

What is the gradient of $\mathbf{z}^h(\mathbf{x})$?

(3)

Review - Backpropagation



$$\delta^h = (\mathbf{W}^o)^T \delta^o \odot \sigma'(\mathbf{z}^h)$$

Compute each part:

$$(\mathbf{W}^o)^T \delta^o = \begin{bmatrix} 1 & -1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} -0.8808 \\ 0.8808 \end{bmatrix} = \begin{bmatrix} (1)(-0.8808) + (-1)(0.8808) \\ (1)(-0.8808) + (2)(0.8808) \end{bmatrix} = \begin{bmatrix} -1.7616 \\ 0.8808 \end{bmatrix}$$

Now ReLU derivative:

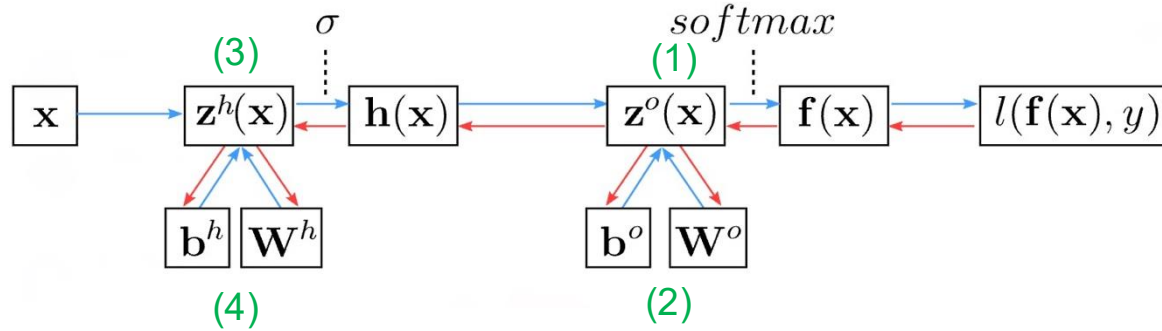
$$\sigma'(\mathbf{z}^h) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Hence:

$$\delta^h = \begin{bmatrix} -1.7616 \\ 0.8808 \end{bmatrix} \odot \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.8808 \end{bmatrix}$$

(3)

Review - Backpropagation



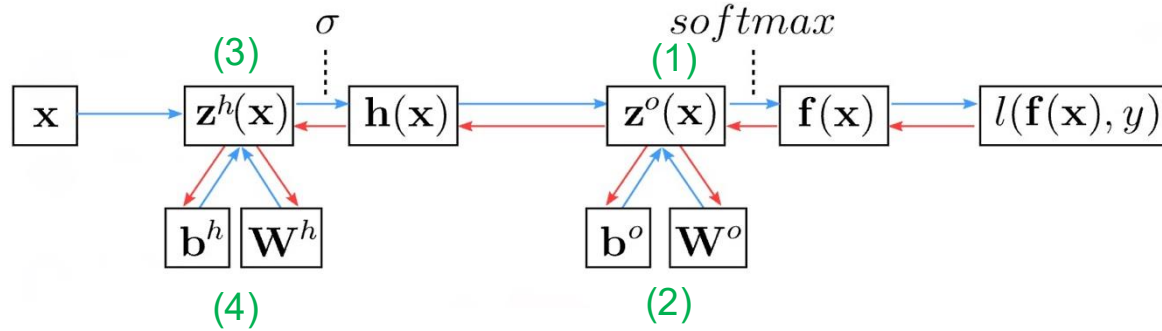
$$\nabla_{W^h} =$$

$$\nabla_{b^h}$$

What is the gradient of hidden weights and bias?

(4)

Review - Backpropagation



$$\nabla_{W^h} = \delta^h x^T, \quad \nabla_{b^h} = \delta^h$$

$$x^T = [1, 2]$$

$$\nabla_{W^h} = \begin{bmatrix} 0 \\ 0.8808 \end{bmatrix} [1, 2] = \begin{bmatrix} 0 & 0 \\ 0.8808 & 1.7616 \end{bmatrix}$$

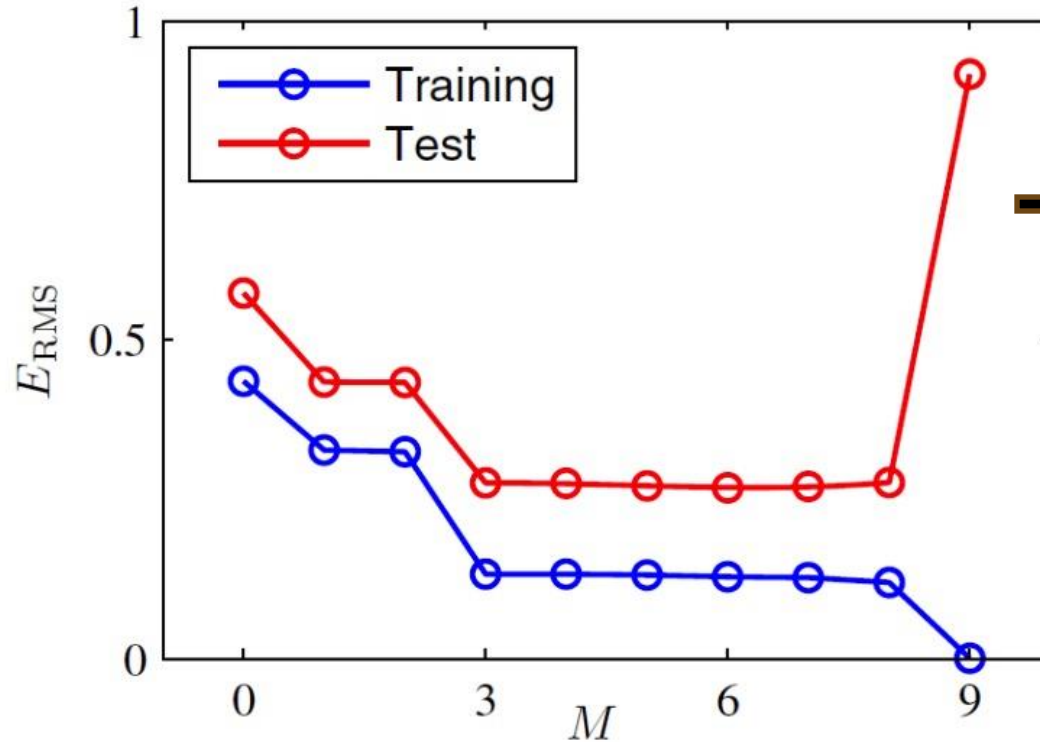
$$\nabla_{b^h} = \begin{bmatrix} 0 \\ 0.8808 \end{bmatrix}$$

(4)

What is regularization?

- In general: any method to prevent overfitting or help the optimization/ generalisation
- Specifically: additional terms in the training optimization objective to prevent overfitting or help the optimisation

Overfitting example



One important reason:
The model is too complex

Figure from *Machine Learning
and Pattern Recognition*, Bishop

Q: Have you ever seen overfitting in your own work?

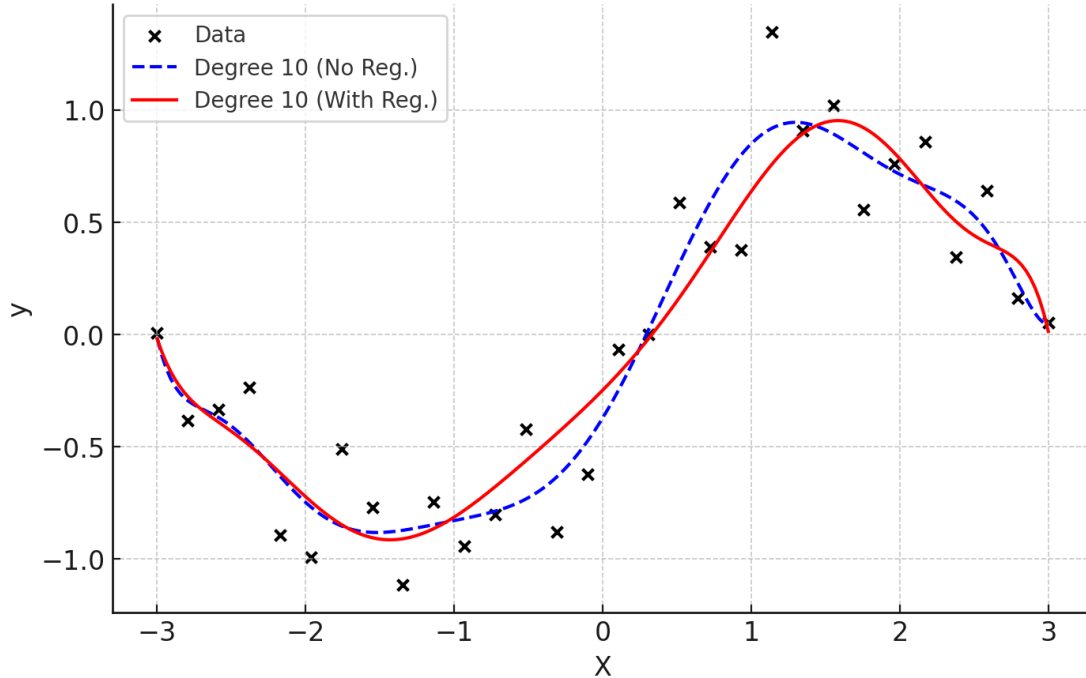
Controlling Model Complexity



- Regularisation is a *penalty* we add to discourage extreme parameter values.
- It's like telling the model: 'Be simpler unless the data really demands complexity.'

Controlling Model Complexity

Polynomial Fit (Degree 10) — With/Without Regularisation



When we add regularisation, the line becomes smoother.

What does this remind you of? Maybe a trade-off?

Regularization as hard constraint

- Example: l_2 regularization

$$\min_{\theta} \hat{L}(\theta) = \frac{1}{n} \sum_{i=1}^n l(\theta, x_i, y_i)$$

$$\text{subject to: } \|\theta\|_2^2 \leq r^2$$

- θ : parameters of the model (e.g., weights and bias)
- x, y : input data and label
- $\hat{L}$: training loss

Regularization as soft constraint

- Example: l_2 regularization

$$\min_{\theta} \hat{L}_R(\theta) = \frac{1}{n} \sum_{i=1}^n l(\theta, x_i, y_i) + \lambda^* \|\theta\|_2^2$$

- θ : parameters of the model (e.g., weights and bias)
- x, y : input data and label
- $\hat{L}$: training loss

Why might soft constraints be easier to handle than hard ones?

Regularization as soft constraint

- Example: l_2 regularization

$$\min_{\theta} \hat{L}_R(\theta) = \frac{1}{n} \sum_{i=1}^n l(\theta, x_i, y_i) + \lambda^* \|\theta\|_2^2$$

- θ : parameters of the model (e.g., weights and bias)
- x, y : input data and label
- $\hat{L}$: training loss

Why might soft constraints be easier to handle than hard ones?

easier optimisation, gradient-based updates.

L2 Regularisation

$$\min_{\theta} \hat{L}_R(\theta) = \hat{L}(\theta) + \frac{\alpha}{2} \|\theta\|_2^2 \leftarrow \text{L2 regularisation term}$$

Training loss

Hyper-parameter to balance
training loss and L2 term

Q: Do you think L2 regularisation leads to sparsity (many zeros)?

L2 Regularisation Example

$$x = [1, 1, 1, 1]$$

$$w_1 = [1, 0, 0, 0]$$

$$w_2 = [0.25, 0.25, 0.25, 0.25]$$

$$w_1^T x = w_2^T x = 1$$

L2 Regularization

$$R(W) = \sum_k \sum_l W_{k,l}^2$$

Which of w_1 or w_2 will the L2 regulariser prefer?

L2 Regularisation Example

$$x = [1, 1, 1, 1]$$

$$w_1 = [1, 0, 0, 0]$$

$$w_2 = [0.25, 0.25, 0.25, 0.25]$$

$$w_1^T x = w_2^T x = 1$$

L2 Regularization

$$R(W) = \sum_k \sum_l W_{k,l}^2$$

Which of w_1 or w_2 will the L2 regulariser prefer?

L2 regularization likes to “spread out” the weights

L1 Regularisation

$$\min_{\theta} \hat{L}_R(\theta) = \hat{L}(\theta) + \alpha ||\theta||_1 \quad \leftarrow \text{L2 regularisation term}$$

Training loss

Hyper-parameter to balance
training loss and L1 term

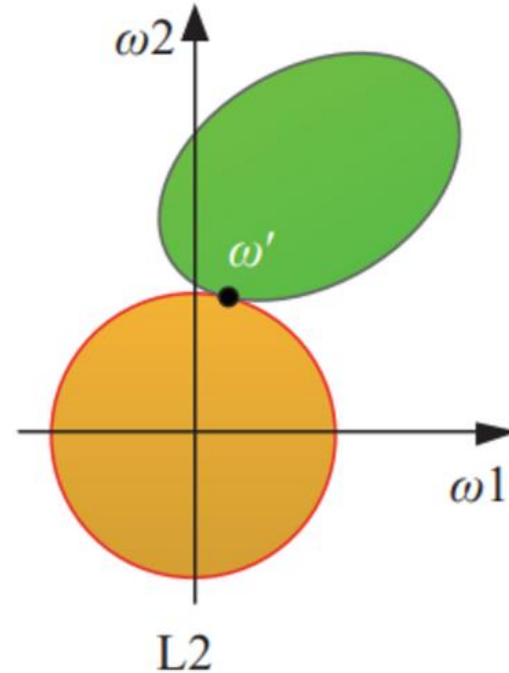
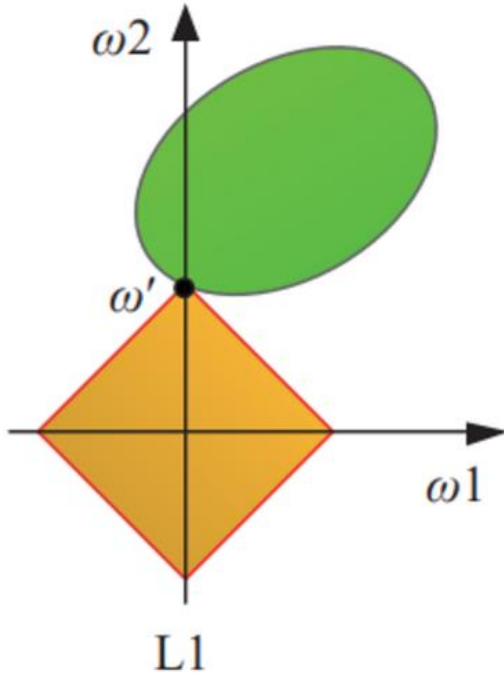
- L1 behaves differently: it pushes some weights exactly to zero.
- This means L1 can perform feature selection: only the most useful features survive.

Why?

L1 and L2

Edge of Green Circle: same Loss value

Prefer to touch
the angle
↓
Some weights
to 0



May touch
anywhere on
the edge
↓
Smooth weights

[Visualisation Video](#)

L1 and L2

Property	L1	L2
Penalises	Absolute value	Squared value
Effect	Some weights = 0	All weights shrink
Result	Sparse model	Smooth model
Use case	Feature selection	Prevent overfitting
Geometry	Diamond	Circle

Both fight overfitting, but in slightly different ways.

Which do you think is more useful in deep networks? L1 or L2?

L1 and L2

Property	L1	L2
Penalises	Absolute value	Squared value
Effect	Some weights = 0	All weights shrink
Result	Sparse model	Smooth model
Use case	Feature selection	Prevent overfitting
Geometry	Diamond	Circle

Both fight overfitting, but in slightly different ways.

Which do you think is more useful in deep networks? L1 or L2?

L1 in Practice

```
for _ in range(epochs):  
    optimizer.zero_grad()  
    yhat = model(x)  
    loss = criterion(yhat, y)  
    if l1_lambda > 0:  
        l1 = How to define the l1 term?  
        loss = How to define the combined loss? # L1 penalty added manually  
    loss.backward()  
    optimizer.step()
```

L1 in Practice

```
for _ in range(epochs):  
    optimizer.zero_grad()  
    yhat = model(x)  
    loss = criterion(yhat, y)  
    if l1_lambda > 0:  
        l1 = sum(p.abs().sum() for p in model.parameters())  
        loss = loss + l1_lambda * l1          # L1 penalty added manually  
    loss.backward()  
    optimizer.step()
```

L2 in Practice

```
opt2 = optim.SGD(m2.parameters(), lr=0.05, weight_decay=1e-2) # L2 via weight_decay
```

```
for _ in range(epochs):
    optimizer.zero_grad()
    yhat = model(x)
    loss = criterion(yhat, y)
    if l2_lambda > 0:
        l2 = How to define the l2 term?
        loss = loss + l2_lambda * l2 # l2 penalty added manually
    loss.backward()
    optimizer.step()
```

L2 in Practice

```
opt2 = optim.SGD(m2.parameters(), lr=0.05, weight_decay=1e-2) # L2 via weight_decay
```

```
for _ in range(epochs):  
    optimizer.zero_grad()  
    yhat = model(x)  
    loss = criterion(yhat, y)  
    if l2_lambda > 0:  
        l2_penalty = lambda_l2 * sum(torch.sum(p ** 2) for p in model_l2.parameters())  
        loss = loss + l2_lambda * l2_penalty # l2 penalty added manually  
    loss.backward()  
    optimizer.step()
```